

broker suitable for learning, testing, and IoT prototyping. It supports standard MQTT clients over TCP on port 1883 and does not require user authentication.

Switchsys MQTT broker details:
Broker URL: mqtt.switchsys.in
Port: 1883 (TCP, non-TLS)

Table 2 shows the MQTT topic structure used in this system for communication between the ESP32-C3 Super Mini and the IoT MQTT Panel application.

Example of a telemetry payload.

```
{
  "cpu_usage": 21.4,
  "ram_usage": 34.7,
  "rssi": -56,
  "chip_temperature": 45.8,
  "last_reset_reason": "POWER_ON",
  "wifi_channel": 1,
  "flash_usage": 23.56
}
```

Working with the Switchsys MQTT broker

The ESP32-C3 Super Mini connects to the local Wi-Fi network and establishes an MQTT session with the Switchsys public MQTT broker. Once connected, the device periodically publishes health and system telemetry while subscribing to MQTT topics for remote control commands.

Telemetry published by the ESP32-C3 super mini.

1. CPU usage (%)
2. RAM usage (%)
3. Wi-Fi signal strength (RSSI in dBm)
4. Chip temperature (°C)
5. Flash memory usage (%)
6. Reset reason
7. Wi-Fi channel (CHx)
8. Device uptime
(Days:Hours:Minutes:Seconds)
9. Remote LED status

Control commands subscribed by the ESP32-C3 super mini.

1. Remote LED control (on/off)
2. Remote ESP32 reset

Software configuration

Before uploading the program to the ESP32-C3 Super Mini, update the

TABLE 3
TYPICAL WI-FI SIGNAL STRENGTH RANGES

RSSI Value (dBm)	Signal Quality
-30	Excellent
-60	Good
-89	Weak

TABLE 4
CHIP TEMPERATURE

Temperature	Condition
20 to 50°C	Normal
50 to 70°C	Warm but safe
>80°C	Risk of instability

```
float getCpuUsage() {
  static char stats[512];
  vTaskGetRunTimeStats(stats);

  char* idlePtr = strstr(stats, "IDLE");
  if (!idlePtr) return -1;

  int idlePercent = 0;
  sscanf(idlePtr, "%s %d %d%%", &idlePercent);
  return 100.0 - idlePercent;
}
```

Fig. 3: Function to calculate CPU usage by measuring the ESP32-C3 idle task runtime

```
float getRamUsagePercent() {
  uint32_t totalHeap = ESP.getHeapSize();
  uint32_t freeHeap = ESP.getFreeHeap();

  float usedPercent = ((float)(totalHeap - freeHeap) / totalHeap) * 100.0;

  if (usedPercent < 0) usedPercent = 0;
  if (usedPercent > 100) usedPercent = 100;

  return usedPercent;
}
```

Fig. 4: Function to calculate RAM usage percentage using ESP32-C3 heap memory

```
int rssi = WiFi.RSSI();
```

Fig. 5: The code used to read the Wi-Fi signal strength (RSSI)

```
float getChipTempAvg(uint8_t samples = 5) {
  float sum = 0;
  for (uint8_t i = 0; i < samples; i++) {
    sum += temperatureRead();
    delay(10);
  }
  return sum / samples;
}
```

Fig. 6: Function to calculate the average on-chip temperature of the ESP32-C3

following parameters in the source code:

1. Wi-Fi SSID and password.

Replace these with the credentials of the local Wi-Fi network.

2. MQTT topic names

(optional). Modify the topic names if required. When using a public MQTT broker, unique topic names are recommended to prevent conflicts with topics used by other users. Appending initials, a system identifier, or another unique suffix to each topic helps avoid naming collisions.

In this system, the code is configured to publish telemetry data, LED status, and uptime information to the MQTT broker while subscribing to topics for remote LED control and device reset commands. Fig. 2 shows the Wi-Fi credentials and MQTT topic configuration defined in the ESP32-C3 source code.

CPU usage (%)

CPU usage indicates the percentage of the ESP32-C3 Super Mini's processing capacity currently being used by running tasks. The ESP32

operates on the FreeRTOS real-time operating system (RTOS), which manages multiple tasks simultaneously.

CPU utilisation is determined by measuring the processor's idle time. Whenever no application tasks

are running, the FreeRTOS idle task executes. By monitoring the runtime of the idle task, the system estimates the percentage of time the processor is idle. The `getCpuUsage()` function retrieves the FreeRTOS runtime statistics, identifies the idle task, determines its idle percentage, and calculates CPU usage using the following equation:

$$\text{CPU usage (\%)} = 100 - \text{Idle Time (\%)}$$

Fig. 3 shows the `getCpuUsage()` function used to calculate ESP32-C3